hydrofrac user guide

Lineage-driven fault injection for Hydro programs, in the simulator

Code: `b169662e51` (branch `hydrofrac-implies`, also `origin/hydrofrac-races`). Written 2026-10-26.

About this guide

This guide describes hydrofrac as the code at commit `b169662e51` behaves. Every printed report in it was produced by running the named test on that commit, and pasted unedited except where it says so (absolute paths are shortened to `target/...`). Claims about why something is true are marked:

- **DEMONSTRATED**: a test shows it; the test is named.
- **ARGUED**: a proof in `docs/design/hydrofrac-soundness.md` (cited as "soundness §n") or `docs/design/hydrofrac-implies.md` ("implies §n"), not a test.
- **ASSUMED**: taken as given about Hydro, its simulator, or your program (soundness §16 lists A1-A16).
- **GUESSED**: my reading, not checked.

Contents:

1. What hydrofrac does
2. The property: one rule, `A ⇒ B`
3. The fault model
4. Writing a check
5. Reading the results
6. Lamport diagrams
7. How the search works
8. How much to trust a certificate
9. Cost
10. Commands

Terms used throughout

- **Node**: a process, or one member of a cluster. Members are named `Receiver@0`, `Receiver@1`, ...; a process by its type name (`Sender`).
- **Collection**: a Hydro stream (or keyed stream) on a node, cluster or process.
- **Record** or **fact**: one element of a collection. Two facts are the same if their `bincode` encodings are equal (§8 explains why that matters).
- **Run**: one complete execution of the program in the Hydro simulator.
- **Schedule**: the choices the simulator makes in one run: which pending batch to release, which snapshot value to show, which order to deliver in, which tick to run next. The simulator's `exhaustive` mode enumerates them.
- **Fault set** (also **hypothesis**): a set of crashes and lost messages to inject into a run.
- **Correct member**: a member whose crash did not happen in that run. A crash planned for "after message m" that never fires (because `m` is never sent) leaves its member correct.
- **Probe**: the failure-free runs, which hydrofrac runs first.
- **Lineage** of a fact: the message deliveries the fact was computed from, read back from the run.
- **Certificate**: hydrofrac's claim that no fault set within the stated bounds, in no schedule, breaks the property.

1. What hydrofrac does

hydrofrac takes a Hydro program that you already test in the simulator, and checks a property of its collections while it injects crashes and lost messages. The property is always one rule: *if a record is in collection `A` (at some members), a matching record is*

in collection `B` (at some members), read at the end of each run. Every collection the program's types mark `EventualConsistency` is checked this way automatically ("every fact some correct member holds, every correct member holds"); you can state other rules with `implies`. The search is Molly's lineage-driven fault injection (LDFI), run on the real program: hydrofrac runs the program without faults, reads off which deliveries each fact came from, asks an integer-programming solver for the smallest fault set that removes every known way of deriving some fact, runs that fault set over the simulator's schedules, and repeats with whatever new derivations it saw. It stops when a run breaks the rule (a **counterexample**, with a Lamport diagram) or when no fault set within the bounds is left that could break it (a **certificate**).

What a verdict means. A verdict is about one rule (one labelled collection, or one `implies` rule) and is one of six kinds (§5). A `Certified` verdict says: for every fault set within the bounds (§3) and every schedule the simulator can take, the rule holds at the end of the run. It is a claim about the simulator's model of your program and its harness, under the assumptions in §8, not about a deployment. A `Counterexample` is a real run of the simulator that breaks the rule; the report says which faults and what is missing. The other four verdicts are not certificates, and `assert_holds` refuses all of them except `Vacuous`.

2. The property: one rule, `A ⇒ B`

2.1 The rule

For collections `A` and `B`, and key functions `ka` (on `A`'s records) and `kb` (on `B`'s):

> If a record `r` is in `A` at a member chosen by the **A-side** setting, then some record `r'` with `kb(r') == ka(r)` is in `B` at the members chosen by the **B-side** setting.

It is evaluated once per run, on the end state: after the faults, the heal and the recovery phase (§3). The keys `ka(r)` of the records the A-side chooses are the run's **obligations**. They are read from each run itself, never copied from another run (`hydrofrac/src/spec.rs`, `evaluate`).

A member's holdings are the union of everything its collection ever emitted during the run: a fact, once emitted, is held to the end. A crashed member holds what it emitted up to and including the step in which it crashed (implies §4; `hydrofrac/src/fault/mod.rs`).

A-side (`hydrofrac::ASide`): which records of `A` are obligations.

setting	obligations
`ASide::Correct`	records held by members that never crash in the run
`ASide::Any`	records held by any member, including what a crashed member emitted up to its crash

B-side (`hydrofrac::BSide`): who must hold a match of each obligation.

setting	the rule is broken if …
`BSide::AllCorrect`	some member of `B`'s location that never crashes lacks a match
`BSide::AtLeast(k)`	fewer than `k` members of `B`'s location that never crash hold a match
`BSide::Some`	no member holds a match, counting what crashed members emitted up to their crash

`AtLeast(k)` counts survivors only: a write stored on `k` replicas that later crash is not durable in a crash-stop model. `Some` keeps the "it existed somewhere" reading, which "no creation" needs (the sender may crash right after sending).

2.2 The two ways to state a rule

Automatically, from types. Any collection on a cluster whose location type carries `EventualConsistency` is a **label**, and becomes the rule `A = B =` that collection, keys the identity, `Correct → AllCorrect`: *every fact a correct member holds, every correct member holds.* You get labels without writing anything:

- `broadcast_closed` over a `fail_stop` or `lossy_delayed_forever` channel types its delivery `EventualConsistency` (the doc comment of `broadcast_closed` in `hydro_lang/src/live_collections/stream/networking.rs`; a plain `lossy` channel gives `NoConsistency`, which is not a label);
- every collection computed from a label by operators that keep the type (`merge_unordered`, `unique` and `values` in the teaching examples) is a label too;

- `assert_has_consistency_of::<Cluster<_, EventualConsistency>>(manual_proof!(..))` makes one by hand.

So the program in §4 has one label, and the echo broadcast has three (`from_sender`, `echo` and `delivered`). Building a fixed protocol on top of a broken hop does not remove the hop's label: in `receivers_that_echo_survive_a_sender_crash` the raw `from_sender` hop is still checked and broken while `delivered`, computed from it, holds (DEMONSTRATED, §5.2). Nor does a later `.weaken_consistency()`: hydrofrac checks a collection where its type is minted (HANDOFF §10; not re-checked for this guide).

Explicitly, with `implies`. The extension trait `hydrofrac::Implies` on any top-level stream (on a process or a cluster, not inside a tick):

```
use hydrofrac::{ASide, BSide, Implies};
a.implies("name", &b, q!(|r| key_of_r), q!(|r| key_of_r), ASide::Correct, BSide::AllCorrect);
```

It adds `a.clone().map(ka)` and `b.clone().map(kb)` beside the program, tagged with the rule's name and settings, consumed by nothing; the program's own dataflow is unchanged (`hydrofrac/src/implies.rs`). The keys must be `Serialize`; the key closures must be `Fn` (no mutation of captures). Two rules with the same name are refused when the program compiles. The rule's verdict is `results.label("name")`.

Bounds differ between the two kinds (§3.4): a label keeps two members of its cluster correct; an `implies` rule does not.

2.3 Instances

The instances the suites check. All five `implies` instances are cross-checked by brute force in `hydro_test/src/cluster/ec_implies_checks.rs` (DEMONSTRATED, test names in the last column).

property	`A`	`B`	keys	A-side ⟶ B-side	test
eventual consistency (a label)	the collection	the same	identity	`Correct → AllCorrect`	every test
regular reliable broadcast (agreement)	deliveries	deliveries	identity	`Correct → AllCorrect`	`echo_broadcast_is_regular_not_uniform`, `urb_is_regular_and_uniform`
uniform reliable broadcast	deliveries	deliveries	identity	`Any → AllCorrect`	the same two
validity	the sender's inputs (a process)	deliveries	identity	`Correct → AllCorrect`	`validity_holds_on_reliable_links_and_breaks_on_lossy_ones`
no creation	deliveries	the sender's inputs	identity	`Any → Some`	`no_creation_holds_and_catches_a_made_up_delivery`
acked ⟹ durable	the client's acks	replicas' stores	the write	`Correct → AllCorrect` or `AtLeast(k)`	`acked_writes_are_durable_only_after_enough_stores`

Regular and uniform agreement differ only in the A-side. The echo broadcast delivers before it forwards, so a member that delivers and then crashes before its forward leaves the others without the item: uniformity breaks with 2 crashes, regularity holds (DEMONSTRATED, `a_member_that_delivers_then_crashes_breaks_uniformity`, report in §5.2).

Validity and no creation relate the same two collections in opposite directions. Validity: every input of a sender that never crashes is delivered at every correct member (`A` is on a process, whose only member is `0`). No creation: every delivery, even at a member that later crashed, was input at some sender, even one that later crashed.

3. The fault model

3.1 The faults

hydrofrac injects faults itself, through a network hook on every channel (`hydrofrac/src/fault/mod.rs`, `FaultHook`). There are three kinds:

- **Lost message**, `omit(m)`: the send `m` is dropped and never delivered. Only on a lossy channel (§3.3).
- **Crash at start**, `crash(n)`: node `n` sends and outputs nothing for the whole run.
- **Crash after a send**, `crash(n after m)`: node `n` performs its sends up to and including `m`, then halts. It sends nothing after `m`. Its local outputs from the step it crashed in are still counted (implies §4); nothing later is.

A message `m` names one send by its content (channel, sender, receiver, payload bytes) and its copy number (how many earlier sends of the same content the run made). So "lose the second of two identical sends" and "crash between two identical sends"

are faults (soundness §0, §17 fix 14). Crash points are named by message, not by position or clock time: hydrofrac never assumes a node sends in the same order under different faults, except where it proved the order (§7.4).

A fault set holds at most one crash per node. A crash point whose message is never sent never fires, and its node counts as correct.

3.2 Phases: the fault phase, the heal, the recovery phase

Every run has the same shape (`drive` in `hydrofrac/src/check/engine.rs`):

1. **Fault phase.** Your `inputs` closure runs: it sends inputs and may wait for outputs. Faults happen only now and while the work it caused settles. A barrier here (`quiesce()`, or sending input after an observation that forced the simulator to quiesce) panics with a harness error, because it would hide every interleaving across it.
2. **Quiesce, then heal.** From the heal on, no message is lost and no new crash fires. Crashed nodes stay crashed.
3. **Recovery phase.** Your optional `recover` closure runs with no faults (`EcCheck::run_then_recover`). This is the place for timer ticks and retry rounds; it may call `quiesce()` between rounds. `EcCheck::run` has an empty recovery phase.
4. **Quiesce**, and evaluate the rule on what the members hold.

The heal is enforced: every run's event log is checked for an omission or a crash after the heal (`EventLog::check_heal`, which panics).

3.3 Which channels lose messages

A channel is lossy, so omissions apply to it, unless it is TCP `fail_stop` (`is_lossy` in `hydrofrac/src/fault/mod.rs`).

- `TCP.fail_stop()`: never loses a message. Only a crash of the sender stops its later sends. This is the "reliable link, crash-stop sender" model.
- `TCP.lossy_delayed_forever()`: any send before the heal may be lost. hydrofrac drops it outright. (In the plain simulator this mode delays a "dropped" message to the end of the execution; under hydrofrac the omission is permanent.) `EcCheck::instrument` turns on the simulator's `test_safety_only()`, which this mode requires.
- Any other mode (plain `lossy`, UDP) is also treated as lossy, but no teaching example uses one, and a plain `lossy` channel gives no `EventualConsistency` label (GUESSED that these behave as above in the search; not tested here).

3.4 How many faults: the bounds

A fault set is **within the bounds** if (soundness §10; `CrashBounds`, `hydrofrac/src/check/engine.rs` `bounds`):

- it holds at most one crash per node;
- it crashes at most `max_crashes` nodes, if you set `EcCheck::new().max_crashes(k)`;
- it crashes at most `nodes − 1` nodes, where `nodes` counts every node of the program;
- **for a label only**: it crashes at most `|members| − 2` members of the label's cluster, so two members stay correct (a lone survivor trivially agrees with itself). `implies` rules have no such cap (implies §5, §10).

Lost messages are **not bounded**.

Two consequences you should know:

- In a 2-member cluster, no member crash is within the bounds for a label. A label's certificate there covers only crashes of nodes outside the cluster (senders, coordinators) and lost messages. This is why the uniform broadcast counterexample needs an `implies` rule rather than a label: it crashes a member.
- `max_crashes` is part of the claim. `Certified` with `max_crashes(1)` says "no single crash breaks it" and nothing about two.

3.5 What is not modelled

- **Crashes or losses during the recovery phase.** Faults stop at the heal. A protocol that needs retries is certified only if its retries run in the recovery phase, and only against faults before it.
- **Recovery of crashed nodes.** Crash-stop: a crashed node never restarts.
- **Real time and timeouts.** There is no clock. Delays are only the reorderings and batchings the simulator's schedules explore. A source driven by time (`Interval`, …) is a "fragile" step (§7.2), and its behaviour is whatever the simulator schedules.
- **Byzantine behaviour.** Nodes run the program; they do not lie, corrupt or duplicate messages.
- **Duplication or corruption by the network**, and partitions beyond individual lost messages.
- **Membership changes.** `with_dynamic_membership`, `with_crashable_process` and `with_crashable_cluster` are refused: `compiled()` panics if a check instrumented the flow (`refuse_sim_faults`; DEMONSTRATED by `the_simulators_own_crashes_are_refused`).

- **Liveness.** Only end states are checked. Nothing certifies that a run makes progress. A wait for an output that never comes just ends the inputs early (but see hole R6, §8.3).
- **The order of facts.** Members are compared on sets of facts, not sequences.
- **Scalars.** A labelled singleton or optional is reported `NotChecked` (§5.1): two members can agree on a set of facts and still disagree on a count or sum over it.

4. Writing a check

4.1 A complete test

This is `a_sender_crash_mid_broadcast_leaves_a_receiver_without_the_item`, verbatim from `hydro_test/src/cluster/ec_teaching.rs` (lines 50-72). The file imports `hydro_lang::live_collections::stream::{ExactlyOnce, NoOrder, TotalOrder}`, `hydro_lang::prelude::*` and `hydrofrac::EcCheck`, and declares `pub struct Sender;` and `pub struct Receiver;`.

```rust
#[test]
fn a_sender_crash_mid_broadcast_leaves_a_receiver_without_the_item() {
    let mut flow = FlowBuilder::new();
    let sender = flow.process::<Sender>();
    let receivers = flow.cluster::<Receiver>();
    let (send, item) = sender.sim_input::<u32, TotalOrder, ExactlyOnce>();
    let received = item.broadcast_closed(&receivers, TCP.fail_stop().name("bcast").bincode());
    let _out = received.sim_cluster_output();

    let mut check = EcCheck::new();
    let sim = check
        .instrument(flow.sim().with_cluster_size(&receivers, 2))
        .compiled();
    let results = check.run(&sim, async || {
        send.send(1);
    });

    println!("{results}");
    let cex = results
        .counterexample()
        .expect("a crash mid-broadcast leaves a receiver without the item");
    assert_eq!(cex.crashes(), 1);
}
```

Line by line:

- `FlowBuilder::new()`, `flow.process::<Sender>()`, `flow.cluster::<Receiver>()`: an ordinary Hydro program with one process and one cluster.
- `sender.sim_input::<u32, TotalOrder, ExactlyOnce>()`: a simulator input. `send` is the test's handle, `item` the stream the program sees.
- `item.broadcast_closed(&receivers, TCP.fail_stop().name("bcast").bincode())`: send each item to every receiver, member 0 first, then member 1, over a reliable channel named `bcast` (the name appears in reports). The result, `received`, is typed `EventualConsistency`, so it is a label, named `received` after its binding.
- `received.sim_cluster_output()`: an ordinary simulator output. hydrofrac does not need it: it taps labelled collections itself (the replicated store in `ec_implies_teaching.rs` has no output and is still checked).
- `EcCheck::new()`: a check with no crash cap.
- `check.instrument(flow.sim().with_cluster_size(&receivers, 2))`: hand the simulation to hydrofrac before compiling it. It adds taps on every label and rule, network hook points, and lineage tracking, turns on `test_safety_only()`, and refuses the simulator's own fault features. Then `.compiled()` as usual.
- `check.run(&sim, async || { send.send(1); })`: the closure is the fault phase (§3.2). It must be deterministic: the same inputs in the same order in every run. `run` returns `Results`.
- `println!("{results}")`: print the report (§5).
- `results.counterexample()`: the first counterexample in report order, if any; `cex.crashes()` counts its crashes.

4.2 The API

On `hydrofrac::EcCheck`:

call	what it does
`EcCheck::new()`	a check with no crash cap
`.max_crashes(k)`	cap the crash sweep at `k`; consumes and returns the check, so `let check = check.max_crashes(2);` re-uses the same compiled `sim` (as `two_redundant_senders_survive_one_crash_but_not_two` does)
`.instrument(sim_flow)`	instrument a `SimFlow`; compile its result
`.run(&sim, inputs)`	run the check with no recovery phase
`.run_then_recover(&sim, inputs, recover)`	run with a recovery phase (§3.2)
`.fragile_sites()`	after compiling: the program's fragile steps (§7.2)
`.cross_check(&sim, inputs, max_faults)`	test-only (feature `cross-check`): the search plus both brute-force oracles; `CrossCheck::assert_agrees()` fails on any disagreement

On `hydrofrac::Results`:

call	what it does
`{results}` (Display)	the report
`.label("name")`	the one rule whose display name is `name`; panics if there is none, or more than one
`.labels_named("name")`	every rule with that name
`.counterexample()`	the first counterexample in report order
`.holds()`, `.assert_holds()`	every rule holds (§5.1); `assert_holds` panics with the report and the reason
`.no_counterexample()`, `.assert_no_counterexample()`	weaker: also accepts sampled searches and spent budgets

On one rule's result, `hydrofrac::check::LabelResult`: `.verdict`, `.stats`, `.name()`, `.counterexample()`, `.holds()` and `.assert_holds()` (same rule as `Results::assert_holds`, panicking with that rule's report only).

On `hydrofrac::check::Counterexample`: `.faults` (the fault set), `.crashes()`, `.missing` (the `(member, fact)` pairs not delivered), `.divergent` (facts some correct members hold that the failure-free run never produced), `.panic` (if the run panicked), `.tolerates` and `.tolerance_certified` (how many crashes were cleared first, and whether that is a certificate), `.diagram` (the path of the focused diagram).

And `hydrofrac::Implies::implies(name, &b, ka, kb, a_side, b_side)` (§2.2).

4.3 How rules are named

A label's display name is, in order of preference (`hydrofrac/src/observe/collections.rs` `ec_name`, `source_names.rs` `site_name`):

1. a name tag on the IR node;
2. read back from your source at the labelling call: the variable the statement binds (`let received = ...`); for a statement that closes a `forward_ref` cycle, the handle's name without `_handle` (`echo_handle.complete(..)` is `echo`); for an unbound `x.assert_has_consistency_of(..)`, `x`; for the first statement or tail of a function, the function's name (`uniform_reliable_broadcast_closed`);
3. the operator's name (`Network`, `Unique`, ...).

Labels made inside library code are named by the library's bindings: `reliable_broadcast_closed` gives `initial`, `echo` and `new_messages` (`hydro_std/src/ec_inference_demos/reliable_broadcast.rs`). The report adds the file, and, when one file has several labels with one name, numbers the later ones: the first has no suffix, the second is `#1`, the third `#2` (each labelled IR node is its own label, soundness §11). An `implies` rule is named by its first argument.

5. Reading the results

5.1 The six verdicts

`hydrofrac::Verdict` (`hydrofrac/src/check/results.rs`). The report line for each is generated by `verdict_line` in the same file; the wording below is quoted from it.

verdict	report says	meaning	`holds` / `assert_holds`
`Certified { tolerates: All }`	`holds: no number of crashes or lost messages breaks it.`	no fault set within the bounds breaks it, in any schedule	accepts
`Certified { tolerates: UpTo(k) }`	`holds: no single crash breaks it (`max_crashes` is 1, so more were not checked).`	the same, with at most `k` crashes (your `max_crashes`)	accepts
`Counterexample`	`broken: ...`	a real run breaks it; the sentence names the faults and what is missing	refuses
`Exhausted`	`no counterexample found, but schedules were sampled, so this is not a certificate (...)`	some fault set had too many schedules (§7.5), so only a sample of them ran	refuses
`Budget`	`not a certificate: the hypothesis budget ran out.`	the search tried 1,024 fault sets (`HYPOTHESIS_BUDGET`) without concluding	refuses
`NotChecked`	`not checked (...), so this is not a certificate.`	a scalar, or a collection inside a tick	refuses
`Vacuous`	`nothing to check (...).`	a single-member label (`a single member agrees with itself`), or nothing was held without faults and faults provably create nothing (`the failure-free run holds no facts`)	accepts

`assert_no_counterexample` is weaker: it refuses only `Counterexample`.

A counterexample sentence also says how far the search got before it found it:

- `No single crash breaks it.` / `No combination of up to k crashes breaks it.`: a certified tolerance. The search raises its crash bound from 0 upwards, so a counterexample with `k + 1` crashes comes with a certificate that no set within the bounds with at most `k` crashes breaks the rule.
- `No counterexample with fewer crashes was found, but schedules were sampled, so that is not a certificate.`
- Nothing more: the counterexample has no crash (lost messages only, or no fault at all: `even the failure-free run`), or exactly one crash with zero crashes certified first (`a single crash of Sender ...`).

Each row is followed by two lines of statistics: fault sets tried (`hypotheses`, counting the failure-free probe), schedules run, wall time; and `races:` how many sets stopped early (§7.3), how many ran one schedule only (§7.4), and whether "claim B" was certified. Claim B is "faults create no disagreement on facts the failure-free run never produced"; it is `certified` on every `Certified` verdict and `not certified` on every other verdict, including counterexamples and `Vacuous` (`RaceNotes::claim_b`).

5.2 Real reports

All from `cargo test -p hydro_test --lib -- cluster::ec_teaching:: cluster::ec_implies_teaching:: --test-threads=1 --nocapture` (10 tests, all passed, 62.5 s), unless the heading says otherwise. Absolute paths shortened to `target/....`

One crash (`a_sender_crash_mid_broadcast_leaves_a_receiver_without_the_item`, the test of §4.1):

```
Receiver:
  [0] received (Stream, ec_teaching.rs:56): broken: a single crash of Sender, right after it sends 1 to Receiver@0 (on
bcast), leaves Receiver@1 without 1.
      diagram: target/hydrofrac/a_sender_crash_mid_broadcast_leaves_a_receiver_without_the_item/
received_ec_teaching.rs/counterexample.svg
      2 hypotheses, 2 schedules, 3.18ms
      races: 0 sets stopped early, 2 sets on one schedule; claim B not certified
```

Read it as: the rows are grouped by location (`Receiver:`); `[0]` is the rule's index; `received` its name, a `Stream` labelled at `ec_teaching.rs:56`. The sender crashed right after its first send, so receiver 1 never got `1`. Two fault sets were tried: the probe (no faults) and the crash.

One lost message (`one_lost_message_leaves_a_receiver_without_the_item`):

```
Receiver:
  [0] received (Stream, ec_teaching.rs:83): broken: losing the message 1 from Sender to Receiver@0 (on bcast) leaves
Receiver@0 without 1.
      diagram: target/hydrofrac/one_lost_message_leaves_a_receiver_without_the_item/received_ec_teaching.rs/
counterexample.svg
      2 hypotheses, 2 schedules, 2.72ms
      races: 0 sets stopped early, 2 sets on one schedule; claim B not certified
```

A fix, and the raw hop it builds on (`receivers_that_echo_survive_a_sender_crash`): `delivered` holds; the single-hop `from_sender` is still a label and still broken.

```
Receiver:
  [0] from_sender (Stream, ec_teaching.rs:150): broken: a single crash of Sender, right after it sends 1 to Receiver@0
(on bcast), leaves Receiver@1 without 1.
      diagram: target/hydrofrac/receivers_that_echo_survive_a_sender_crash/from_sender_ec_teaching.rs/
counterexample.svg
      2 hypotheses, 2 schedules, 4.44ms
      races: 0 sets stopped early, 2 sets on one schedule; claim B not certified
  [1] echo (KeyedStream, ec_teaching.rs:158): holds: no number of crashes or lost messages breaks it.
      3 hypotheses, 3 schedules, 6.10ms
      races: 0 sets stopped early, 3 sets on one schedule; claim B certified
  [2] delivered (Stream, ec_teaching.rs:154): holds: no number of crashes or lost messages breaks it.
      3 hypotheses, 3 schedules, 5.38ms
      races: 0 sets stopped early, 3 sets on one schedule; claim B certified
```

So `results.assert_holds()` would fail on this program; the test asserts `results.label("delivered").assert_holds()`. Note what "no number of crashes" means here: with 2 receivers, no receiver crash is within a label's bounds (§3.4), so this certificate is about crashes of `Sender` and lost messages (none: the channels are `fail_stop`).

A crash cap (`two_redundant_senders_survive_one_crash_but_not_two`; first `max_crashes(1)`, then `max_crashes(2)` on the same `sim`):

```
Receiver:
  [0] received (KeyedStream, ec_teaching.rs:190): broken: a single crash of Sender@0, right after it sends 1 to
Receiver@0 (on bcast), leaves Receiver@1 without (Sender@0, 1).
      diagram: target/hydrofrac/two_redundant_senders_survive_one_crash_but_not_two/received_ec_teaching.rs/
counterexample.svg
      2 hypotheses, 2 schedules, 3.98ms
      races: 0 sets stopped early, 2 sets on one schedule; claim B not certified
  [1] items (Stream, ec_teaching.rs:191): holds: no single crash breaks it (`max_crashes` is 1, so more were not
checked).
      1 hypothesis, 1 schedule, 3.00ms
      races: 0 sets stopped early, 1 set on one schedule; claim B certified

Receiver:
  [0] received (KeyedStream, ec_teaching.rs:190): broken: a single crash of Sender@0, right after it sends 1 to
Receiver@0 (on bcast), leaves Receiver@1 without (Sender@0, 1).
      diagram: target/hydrofrac/two_redundant_senders_survive_one_crash_but_not_two/received_ec_teaching.rs-2/
counterexample.svg
      2 hypotheses, 2 schedules, 2.55ms
      races: 0 sets stopped early, 2 sets on one schedule; claim B not certified
  [1] items (Stream, ec_teaching.rs:191): broken: a crash of Sender@0, right after it sends 1 to Receiver@0 (on
bcast), and a crash of Sender@1, right after it sends 1 to Receiver@0 (on bcast), leave Receiver@1 without 1. No
single crash breaks it.
      diagram: target/hydrofrac/two_redundant_senders_survive_one_crash_but_not_two/items_ec_teaching.rs/
counterexample.svg
      2 hypotheses, 2 schedules, 2.74ms
      races: 0 sets stopped early, 2 sets on one schedule; claim B not certified
```

`items` was certified at one crash after a single run (`1 hypothesis`): each receiver's `1` has two robust derivations, one per sender, and no single crash kills both, so the solver had no fault set to propose (§7.3).

`implies`: regular holds, uniform breaks (`a_member_that_delivers_then_crashes_breaks_uniformity`). Rows 0-2 are the labels inside `reliable_broadcast_closed`; rows 3-4 are the two rules.

```
Member:
  [0] initial (Stream, reliable_broadcast.rs:133): broken: a single crash of Source, right after it sends 1 to
```

```
Member@0 (on initial), leaves Member@1 without 1.
      diagram: target/hydrofrac/a_member_that_delivers_then_crashes_breaks_uniformity/initial_reliable_broadcast.rs/
counterexample.svg
      2 hypotheses, 2 schedules, 5.65ms
      races: 0 sets stopped early, 2 sets on one schedule; claim B not certified
  [1] echo (KeyedStream, reliable_broadcast.rs:159): holds: no number of crashes or lost messages breaks it.
      3 hypotheses, 3 schedules, 7.02ms
      races: 0 sets stopped early, 3 sets on one schedule; claim B certified
  [2] new_messages (Stream, reliable_broadcast.rs:152): holds: no number of crashes or lost messages breaks it.
      3 hypotheses, 3 schedules, 6.30ms
      races: 0 sets stopped early, 3 sets on one schedule; claim B certified
  [3] regular (Stream, implies, Correct → AllCorrect): holds: no number of crashes or lost messages breaks it.
      5 hypotheses, 5 schedules, 8.75ms
      races: 0 sets stopped early, 5 sets on one schedule; claim B certified
  [4] uniform (Stream, implies, Any → AllCorrect): broken: a crash of Member@0, right after it sends 1 to Member@0 (on
echo), and a crash of Source, right after it sends 1 to Member@0 (on initial), leave Member@1 without 1. No single
crash breaks it.
      diagram: target/hydrofrac/a_member_that_delivers_then_crashes_breaks_uniformity/uniform_implies.rs/
counterexample.svg
      4 hypotheses, 4 schedules, 8.05ms
      races: 0 sets stopped early, 3 sets on one schedule; claim B not certified
```

`regular` searched 5 fault sets (it may crash members, since it is not a label), `new_messages` only 3 (no member crash is allowed for a label of a 2-member cluster).

`AtLeast(1)` under a cap (`two_copies_survive_one_crash_but_not_two`, 3 replicas, the client acks after 2 stores). The `stored` label (the raw write broadcast, over a lossy channel) is broken by a lost write in both runs; only the rule matters here:

```
Source:
  [1] durable (Stream, implies, Correct → AtLeast(1)): holds: no single crash breaks it (`max_crashes` is 1, so more
were not checked).
      12 hypotheses, 12 schedules, 19.24ms
      races: 0 sets stopped early, 12 sets on one schedule; claim B certified
...
Source:
  [1] durable (Stream, implies, Correct → AtLeast(1)): broken: a crash of Member@0, right after it sends 1 to Source
(on ack), and a crash of Member@1, right after it sends 1 to Source (on ack), and the loss of the message 1 from
Source to Member@2 (on write) leave Member@2 without 1. No single crash breaks it.
      diagram: target/hydrofrac/two_copies_survive_one_crash_but_not_two/durable_implies.rs/counterexample.svg
      15 hypotheses, 15 schedules, 22.68ms
      races: 0 sets stopped early, 15 sets on one schedule; claim B not certified
```

(The `...` replaces the second run's `Member:` block, identical in kind to the first.) The rule is grouped under `Source:`, the location of its `A` side; the missing member is named in `B`'s location. The diagram directory says `implies.rs` because the rule's key nodes are built inside `hydrofrac/src/implies.rs`.

`Vacuous` (from `hydro_test`'s `brute_force_agrees_on_a_plain_broadcast`, in `cluster::ec_bruteforce`):

```
  [1] echoes (Stream, reliable_broadcast.rs:60): nothing to check (the failure-free run holds no facts).
      1 hypothesis, 1 schedule, 3.08ms
      races: 0 sets stopped early, 1 set on one schedule; claim B not certified
```

`NotChecked` (from `review_c_a_singleton_label_is_not_a_certificate`, `cargo test -p hydrofrac --features cross-check`), and what `assert_holds` panics with:

```
  [0] mine (Singleton, review_c_tests.rs:267): not checked (Singleton (not a set-valued collection)), so this is not a
certificate.
hydrofrac: label `mine (review_c_tests.rs)` is not checked (Singleton (not a set-valued collection)), so it is not
certified
```

`Exhausted` (from `review2b_a_sampled_miss_is_not_accepted_by_assert_holds`, the program past the schedule budget), with `assert_holds`'s panic:

```
hydrofrac: schedules were sampled, so the search of label `arrived (review2b_tests.rs)` is not a certificate
(`assert_no_counterexample` accepts it)
cluster:
```

```
  [0] arrived (Stream, review2b_tests.rs:178): no counterexample found, but schedules were sampled, so this is not a
certificate (searched up to 0 crashes; schedule choices: batch at review2b_tests.rs:179, batch at
review2b_tests.rs:186).
      1 hypothesis, 3342 schedules, 6.91s
```

The schedule count includes every schedule enumerated before the budget ran out, plus the 8 samples (the counter in `races_engine.rs` counts both). The "schedule choices" are the program's nondeterministic steps (§7.2).

`Budget`: I found no test in the three suites that prints one, so there is no real report to quote.

6. Lamport diagrams

6.1 Where they land

Every counterexample writes two diagrams (`write_diagrams` in `hydrofrac/src/check/engine.rs`; `hydrofrac/src/lamport/out.rs`) into

```
<target>/hydrofrac/<test>/<label>/
```

- `<target>` is `$CARGO_TARGET_DIR` if set, else the ancestor directory of the test executable that holds Cargo's `CACHEDIR.TAG` (normally the workspace `target/`), else `target`.
- `<test>` is the last `::` segment of the test thread's name, the test function's name.
- `<label>` is the rule's display name and file, with every run of characters other than letters, digits, `-`, `_` and `.` turned into one `_`: `received (ec_teaching.rs)` becomes `received_ec_teaching.rs`. If the same process already wrote that directory (a second `run` of the same label in one test), the next one gets `-2`, `-3`, …

Each directory holds:

- `counterexample.dot` and `counterexample.svg`: the **focused** diagram. The report's `diagram:` line names it.
- `counterexample-full.dot` and `counterexample-full.svg`: the **full** diagram of the same run.

The `.svg` files are written only if Graphviz's `neato` is on the `PATH`; otherwise only the `.dot`, and the `diagram:` line names the `.dot`. Render it by hand with `neato -n2 -Tsvg counterexample.dot -o counterexample.svg` (the `-n2` matters: every position is pinned in the file).

6.2 Focused and full

The **full** diagram shows the whole failing run: every message, every output of every tapped collection (decoded), and the raw bytes (first 8, in hex) of other outputs such as `sim_cluster_output` ports.

The **focused** diagram (`hydrofrac/src/lamport/focus.rs`) keeps:

- the sends and outputs of the broken facts (the missing or divergent ones), and every delivery in their lineage;
- every message a fault took (lost, or not sent because its sender had crashed), and every message a fault names;
- the crashes and the heal;

and then only the outputs of this rule's own collections. It drops every other message and output; a node left with nothing loses its lane. Its title is the report's sentence, then a caption counting what was left out, then a `faults:` line in compact notation, then the legend. If the run panicked, no fact is broken, and the whole run is drawn.

6.3 Reading one

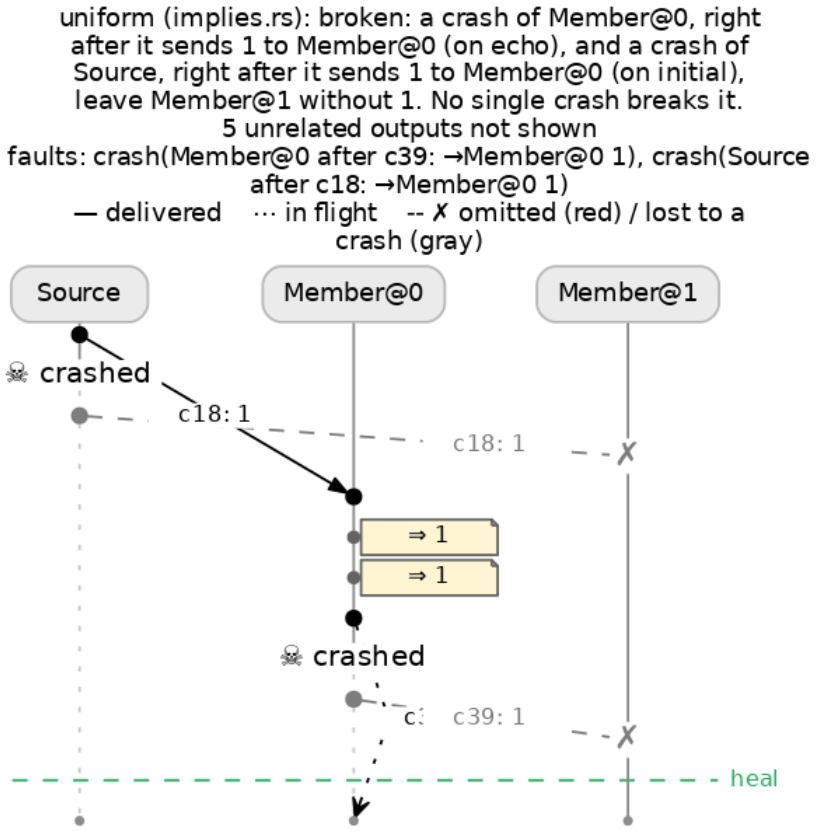


Figure 1: Focused diagram of `uniform` in `a_member_that_delivers_then_crashes_breaks_uniformity`, rendered with `neato -n2 -Tpng` from `counterexample.dot`. The only change: its one-line title is wrapped to fit the page.

This is the focused diagram of the `uniform` rule from §5.2, at `target/hydrofrac/a_member_that_delivers_then_crashes_breaks_uniformity/uniform_implies.rs/counterexample.dot`.

- **Lanes.** One vertical line per node (`Source`, `Member@0`, `Member@1`), time flowing down. Each event has its own row, in the order of the run's event log (`dot.rs`: "each send, receive, output, crash and the heal gets its own rank, in program order"). Distances mean nothing.
- **Title.** The verdict sentence; `5 unrelated outputs not shown`; then `faults: crash(Member@0 after c39: →Member@0 1), crash(Source after c18: →Member@0 1)`. In the diagram a message is labelled by its channel's `.name(..)` if it has one, else `c<id>`: `reliable_broadcast_closed` names neither channel, so `c18` is the initial broadcast and `c39` the echo (the text report calls them `initial` and `echo`; the two use different naming code, `lamport/mod.rs` and `report/mod.rs`).
- **Source.** It sends `1` to `Member@0` (a solid black arrow, delivered), then ☠ `crashed`. The send it would have made next, to `Member@1`, is drawn grey and dashed, ending in a grey ✗ on `Member@1`'s lane: lost to the crash. Below the crash its lane is dotted.
- **Member@0.** It receives `1`, and two yellow notes ⇒ `1` show its outputs: the rule's `A` and `B` key taps both read the same `delivered` stream. Then it sends the echo `1` to itself, which is dotted (`in flight`: sent but never received, because `Member@0` crashed right after sending it; its label `c39: 1` is partly covered by the next one), and crashes. Its echo to `Member@1` is lost to the crash (grey, ✗).
- **Member@1.** Nothing arrives; it never outputs `1`.
- **The heal** is the green dashed line near the bottom. Nothing happens after it here.

Read as a sentence: `Member@0` delivered `1` (so under `ASide::Any` it is an obligation), and `Member@1`, which never crashes, never got it from anyone. The legend at the top gives the line styles: solid delivered, dotted in flight, dashed with ✗ omitted (red) or lost to a crash (grey). A lost message (as in `one_lost_message_leaves_a_receiver_without_the_item`) is drawn red and dashed.

The full diagram of the same run has 7 output notes rather than 2: six decoded `⇒ 1` outputs (the taps of every label and rule on `Member@0`'s collections) and `⇒ 01000000`, the raw bytes of the `sim_cluster_output` port.

7. How the search works

This section is a summary. The full argument is soundness §1-§14; the code is `hydrofrac/src/check/races_engine.rs` (the loop), `hydrofrac/src/races/` (clauses, the integer program, the sweep) and `hydrofrac/src/provenance/` (lineage and the fragile-step analysis).

7.1 Lineage

`EcCheck::instrument` rewrites the program so that every record carries a tag: the message deliveries it was computed from, and the *fragile steps* (§7.2) it passed through. Operators the rewrite cannot see into are *opaque islands*; their records fall back to a coarser rule, count as fragile, and are listed at the end of the report (`opaque to provenance (execution rule): ...`).

From each run the search reads, for every **pair** (a member of `B`'s location, an obligation key), the derivations of that key at that member: one **clause** per tap emission, listing the deliveries it needed (with, for each, whether its channel is lossy and who sent it) and the fragile steps it passed (its *premises*). A clause with no premise is **robust**.

A fault **kills** a clause if: it omits one of its lossy deliveries; it crashes the sender of one of them at start; or it crashes that sender after some message `m`, unless it is proved that the delivery is sent before `m` (§7.4). The order a run happened to show is never used to decide a kill (soundness §5).

7.2 Robust and fragile steps; schedule choices

A step is **robust** if the set of records it emits over a run depends only on the set of records it receives over the run, whatever the batching and order, and does not depend on whether a later tick runs. Every other step is **fragile**. The classification is an allowlist (`hydrofrac/src/provenance/fragile.rs`; the table in soundness §3): `map`, `filter`, `flat_map`, `merge`, `unique`, networks, top-level joins, top-level sources and harness inputs are robust (closures must also pass a purity allowlist, `provenance/purity.rs`); snapshots, folds, reduces and scans, joins and anti-joins inside a tick, `defer_tick`, constants inside a tick, time-driven sources, a batch into a tick that also snapshots a singleton, and every kind not listed are fragile. Two shapes are exempt by proof: the `filter_not_in` + `defer_tick` deduplication idiom, and the trusted `quorum` (§7.4).

Why it matters (Molly's question, answered differently). Lemma R (soundness §2, ARGUED): if every delivery of a *robust* derivation happens in a run, the derivation's fact is held in that run (or its member crashed), in every schedule. So a robust clause that a fault set does not kill protects its pair under that fault set, and the search may skip the set. A fragile clause never protects anything; it only steers the search toward sets worth trying.

Separately, a **schedule choice** is a step where the simulator may branch (a `batch`, a snapshot, an `observe_nondet`, a fold over an unordered input, an impure closure, ...; `hydrofrac/src/provenance/choices.rs`, also an allowlist). The report lists them with a sampled verdict.

7.3 The loop

1. **Probe.** Run the program with no faults, over every schedule (or one, §7.4). Read every pair's clauses. The probe's obligations are the *domain*.
2. **Candidates.** Every fault some run revealed: the loss of each lossy message sent before the heal, a crash at start of each node, a crash of each node after each message it sent before the heal, and (unless the program is fragile-free, which proves the probe shows every send) one `Unseen(n)` per node: "crash `n` after a message no run showed", which kills everything `n` sends.
3. **Propose.** The crash bound `b` starts at 0. An integer program (the `good_lp` crate with its pure-Rust `microlp` solver, `hydrofrac/src/races/ilp.rs`) proposes a fault set within the bounds, with at most `b` crashes, that is **admissible** for some pair: it kills every robust clause of that pair and does not excuse it (a crash at start of the pair's member, or a set already shown to leave the fact *dead*, excuses it). It proposes the cheapest first, and first by Molly's heuristic (kill every known clause of a pair, robust or not), then by the admissibility rule alone. Every tried set is excluded thereafter. For `AtLeast(k)` a key needs `|B| − k + 1` broken pairs, and any crash of a member breaks its pair (implies §3).
4. **Run** the proposal over its schedules. If a run breaks the rule (or panics), report a counterexample. Otherwise add the new clauses and candidates. An exploration **stops early** once the clauses learned so far make the set inadmissible: the remaining schedules cannot break the rule (the corollary of Lemma R).
5. **Conclude** at bound `b` when the program has no solution. Raise `b`. The verdict is `Certified(UpTo(k))` when `b` reaches your `max_crashes`, `Certified(All)` when `b` reaches `nodes − 1` or every candidate crash is used. If new candidates appear after a bound concluded, the sweep starts again from 0 (tried sets are kept).

6. **Completeness sweep** ("claim B"). Steps 3-5 cover the probe's obligations. Faults can create new ones (a retry, a failover). Unless the program is fragile-free or choice-free (§7.4), where that is proved impossible, the search then runs **every** fault set within the bounds over the candidates, every schedule, no early stop, before it certifies. That is where certificates of programs with timers and batching get expensive.

The search gives up after 1,024 fault sets per rule (`HYPOTHESIS_BUDGET`): `Budget`.

7.4 The shortcuts

Each is ARGUED in the soundness document. Each saves runs; none weakens what a certificate claims.

- **Fragile-free program** (no fragile step anywhere; `.fragile_sites()` is empty): every derivation is robust; the probe shows every message any run can send, so no `Unseen` candidate is needed (soundness §6.2); faults create no new fact, so claim B needs no sweep (§13); an empty domain is `Vacuous` with no search (also in a choice-free program, §11); and a set can be shown to make a fact *dead*, which excuses it in larger sets (§8).
- **Choice-free program** (no schedule choice): each fault set has one schedule; derivations that used only first copies of messages are treated as robust (§3.1); claim B needs no sweep (§13, given A13: such a program is monotone).
- **One schedule** (§14): a set runs one schedule, not all, in a choice-free program when its crashes are all at start, or in a fragile-free program when every fault of the set is *exact* (it provably cuts the same sends in every schedule). Not for a set with a crash point on `A`'s location under `ASide::Any`, or on `B`'s under `BSide::Some` (implies §5, §14). The `races:` line counts these sets.
- **Order proofs** (§6-§7), used to decide exactly which sends a crash point cuts:
  - `broadcast_closed` sends each payload to members `0, 1, ..., n−1` in that order, recognised by its exact compiled IR shape (§6.1);
  - a *sequenced* closed broadcast, whose data is directly a `TotalOrder`, `ExactlyOnce` harness input of a process, sends whole payload blocks in the harness's order (§6.4), checked against every run the search executes;
  - the trusted `hydro_std::ec_inference_demos::quorum::quorum`, in its exact compiled form, emits its result in every run where `threshold` distinct attestations arrive, whatever the batching, with lineage exactly those attestations (§7). A hand-written copy is not trusted (DEMONSTRATED, `hand_written_quorum_stays_fragile`).

7.5 The schedule budget

There is no setting for exploration (soundness §14; `explore_schedules` in `hydrofrac/src/check/engine.rs`):

- each fault set is explored over every schedule until those schedules have sent **40,000 messages** in total (`SEND_BUDGET`). If the enumeration finishes first, the set was explored exhaustively;
- otherwise the enumeration stops and **8 seeded schedules** run instead (`SAMPLED_SCHEDULES`, through `CompiledSim::sample_then`: schedule `i` draws its choices from a fixed pseudo-random stream seeded by `i`, so every run of the check repeats). DEMONSTRATED deterministic by `sim_sample_then_is_seeded`;
- a rule whose search sampled anything is never `Certified` or `Vacuous`, only `Exhausted` at best; a counterexample it finds is real, but its tolerance is not certified. A sampled probe makes every rule's search sampled. Once one set of a rule is sampled, the rest of that rule's search samples at once, without enumerating first.

The budget counts messages, not schedules, so that it bounds time (the comment on `SEND_BUDGET`: a G-Set gossip schedule sends about 45 messages and costs 35-85 ms; a CTP-2 one about 12 and 4-7 ms). Setting `BOLERO_RANDOM_TEST_TIME_MS` makes the check refuse to run, because bolero would silently cut "exhaustive" enumeration short (DEMONSTRATED, `a_time_bounded_exhaustive_run_is_refused`).

8. How much to trust a certificate

Short answer: a certificate is strong evidence within the fault model and the bounds, and it has known holes. Treat it as "no counterexample exists *unless* one of the holes in §8.3 applies to your program". The handoff note (`docs/design/HANDOFF-hydrofrac-races.md`) warns that three review rounds found real false certificates, that most are fixed and tested, and that the fixes have had no second adversarial review. That warning still applies to this commit.

8.1 DEMONSTRATED: brute-force cross-checks

Two oracles check the search's verdicts by enumeration (test-only, feature `cross-check`; `EcCheck::cross_check`):

- **brute force** (`hydrofrac/src/check/brute_force.rs`): every fault set within the bounds with at most `max_faults` faults (every cross-check file in the suites sets `MAX_FAULTS = 2`; a few calls pass 1), over every schedule, with no lineage and no pruning. It shares the simulator, the instrumented program, the taps, the fault hook and the bounds with the search.

- **oracle 2** (`hydrofrac/src/check/oracle2.rs`): shares as little as possible: it compiles its own fork of your flow, with its own taps (no provenance rewrite), its own fault hook and its own evaluation of the rule. It shares the simulator, your IR, your harness and the definition of the bounds.

They agree with the search, in every test below, on: the kind of verdict; for a certificate at `k` crashes, that no enumerated set with at most `k` crashes breaks the rule; for a counterexample after a certified tolerance `t`, that no enumerated set with at most `t` crashes breaks it and the counterexample has `t + 1` (soundness §15). All of these passed on this commit (§9):

- `hydrofrac/src/check/brute_tests.rs`, 27 tests: plain, lossy and echo broadcasts, relays, an ownership exchange, provenance precision, the two lite unsoundnesses (`suspected_total_loss_masks_a_partial_loss_*`, `suspected_positional_crash_order_*`), every regression for a fixed false certificate (soundness §17: e.g. `tick_constants_are_fragile`, `one_labelling_line_twice_is_two_labels`, `a_fact_only_faults_create_is_checked`, `impure_closures_are_fragile_choices`), and the shortcuts (`the_choice_free_shortcut_runs_one_schedule_and_aims_the_search`, `a_sequenced_broadcast_runs_each_crash_point_of_its_sender_once`);
- `hydrofrac/src/check/review2a_tests.rs`, `review2b_tests.rs`, `review_c_tests.rs`: regressions for fixed review findings;
- `hydrofrac/src/check/fanout_tests.rs`: the order proofs against every schedule of every set of at most two faults (`sequenced_broadcasts_keep_the_harness_order_under_every_fault_set`, which a deliberate mutation fails);
- `hydro_test/src/cluster/ec_teaching_checks.rs` and `ec_implies_checks.rs`: every program and verdict in this guide, five tests each;
- `hydro_test/src/cluster/ec_bruteforce.rs`: `reliable_broadcast_closed` and plain broadcasts with 1-3 payloads, `ask_once`, two-phase commit with cooperative termination (CTP), echo broadcast over two members;
- `hydro_test/src/cluster/ec_races_targets.rs`: uniform broadcast over 2 members (`races_targets_urb2_threshold1`, `_threshold2`), CTP-2 (`races_targets_ctp2_timers_after_the_faults`, `races_targets_ctp2_racing_sampled`);
- `hydro_test/src/cluster/ec_two_pc_hydrofrac.rs`: two-phase commit and CTP variants;
- `hydro_test/src/cluster/ec_quorum_trust.rs`: the trusted quorum (`races_planted_local_attestation`).

What "demonstrated" does **not** cover:

- **More than two faults.** Both oracles stop at `max_faults = 2`. A certificate's claim about 3 or more faults rests on the argument alone. A counterexample with more than two faults is only re-run, not matched against the enumeration (`disagreement` in `brute_force.rs`): the 3-fault `AtLeast(1)` counterexample of §5.2 is confirmed by re-running it, and its "no single crash breaks it" by enumerating every set of at most two faults with at most one crash.
- **Programs no oracle finishes**, which rest on the argument: uniform broadcast over 3 members, echo broadcast over 3 members, ABD, G-Set gossip, multi-Paxos (soundness §15, "Argument-only").
- **Common-mode bugs.** A bug in the simulator or the harness hits both oracles. A bug in the taps, the fault hook or fact identity hits the search and brute force; oracle 2 exists because the two fatal bugs of review 3 (merged labels, silently truncated "exhaustive" runs) were shared by the search and brute force (HANDOFF §8).

8.2 ARGUED: the soundness document

The certificate theorem (soundness §1) rests on these, each proved on paper, none mechanically checked:

- Lemma R: a robust derivation whose deliveries all happen yields its fact (or its member crashed) in every schedule (§2);
- the fragile-step table matches what DFIR and the simulator do (§3; "not proved", checked only by the cross-checks);
- the order proofs: fan-out order (§6.1), sequenced broadcasts (§6.4; tested against brute force on a small program, but the one corpus verdict it changed, in the uniform-broadcast demo, has no cross-check), trusted `quorum` (§7, by code reading of `hydro_std/src/ec_inference_demos/quorum.rs`);
- dead facts (§8), completion of the fault universe (§9), the shortcuts' claim B (§13), one-schedule exactness (§14), copy multiplicity in fragile-free programs (§6.2, argued, not checked);
- panics: a pruned set never hides a panic within the bounds (§18, argued);
- the `implies` generalisation of each of these (implies §5).

And on the assumptions A1-A16 (soundness §16), of which the ones you can break from your own program are: **A3**, closures are pure (an allowlist checks names, not what they resolve to); **A4**, `Serialize` is injective on labelled values; **A8**, the harness is deterministic and does not branch on what it observes.

8.3 KNOWN OPEN HOLES

Each can produce a wrong certificate or a missed failure on this commit. None is fixed.

hole	plain English	pinned by
R1	A "this delivery cannot be lost" conclusion drawn from a run where the send happened *after* the heal is reused in a run where the same send happens *before* the heal, where it can be lost	`review3b_a_post_heal_copy_protects_a_pre_heal_loss`, branch `hydrofrac-review3-b`
R2	The same hole, with a break that needs 3 faults, so both oracles (capped at 2) miss it too	`review3b_a_three_fault_break_beyond_both_oracles`, branch `hydrofrac-review3-b`
R3	The purity allowlist is fooled by a counter kept outside the closure (an `Rc<Cell<_>>` from a source expression, read and replaced with allowed method names)	`review3a_a_shared_cell_counter_is_known_pure`, branch `hydrofrac-review3-a`
R4	The purity allowlist is fooled by a user function passed by name (`Some(m).map(next_seq)`) rather than called	`review3a_a_helper_passed_as_a_value_is_known_pure`, `review3a_the_purity_allowlist_accepts_hidden_state`, branch `hydrofrac-review3-a`
R5	A label on a single-member cluster is `Vacuous` and never run under faults, so a panic only faults cause (here, in the recovery phase) is missed	`review3a_a_single_member_label_hides_a_fault_panic`, branch `hydrofrac-review3-a`
R6	Inputs the harness sends *after* waiting for an output are assumed present in every run, but a fault can stall the wait so they are never sent	`review3b_an_input_sent_after_a_wait_is_not_in_every_run`, branch `hydrofrac-review3-b`
`Serialize` that drops data	Facts are compared by their `bincode` bytes. A `#[serde(skip)]` field or a lossy custom `Serialize` makes different values one fact, which hides a disagreement on the dropped data. Neither oracle can see it (both compare bytes)	no test; soundness §11, A4, §18
two oracles, two faults	No cross-check reaches 3 faults; R2 is a real instance	`review3b_a_three_fault_break_beyond_both_oracles`
sampled oracles on racing CTP	The racing-timer CTP cross-checks have too many schedules for exhaustive oracles (the CTP-2 failure-free run alone passed 31,957 schedules in 10 minutes, soundness §15), so both oracles sample 64 seeded schedules per set (`RACING_ORACLE_SAMPLES`). Those cross-checks miss what the sample misses	`ctp2_one_shot_timer_breaks_at_one_coordinator_crash` and the other racing tests in `ec_two_pc_hydrofrac.rs`, `races_targets_ctp2_racing_sampled`

Each pinning test asserts the wrong behaviour, so it *passes* while the hole is open (for most: the search certifies while an oracle breaks the rule; for R2, while a hand-written three-fault set breaks it). The round-3 branches are local and not merged; they were written against `hydrofrac-races-int` (`56effda043`), an ancestor of this commit, and no later commit claims to fix them (checked with `git log hydrofrac-races-int..b169662e51`). I did not re-run them for this guide.

The `unique` two-path issue in `implies` (implies §3, "No preserve rule"). Lineage keeps one derivation per tap emission. A record reached by two paths and then deduplicated (`unique`, the dedup idiom) shows only one. Molly may skip a fault set that kills every known derivation of a precondition, because Molly's provenance holds every derivation; in hydrofrac that rule would be unsound: killing the one visible path does not remove the record, so skipping the set could certify wrongly. **hydrofrac therefore has no preserve rule**: only proved absence (crashes at start, dead sets) exempts an obligation. So this is not an open hole in the code; it is an ARGUED reason for a design restriction, with no test and no branch. The cost side is real: such records make the search try fault sets that change nothing.

Smaller known gaps (soundness §18): no faults during recovery; liveness; schedule-exploration gaps of the simulator itself (`docs/design/sim-exhaustive-misses-abd-kv.md`, ABD-KV); the order of facts is not compared; scalars and collections inside a tick are not checked; a stuck tick is the simulator's semantics, not necessarily a deployment's.

9. Cost

9.1 Suite times on this commit (measured 2026-10-26)

Debug build, already compiled, `--test-threads=4`, one run each on the machine this guide was written on, with other work running beside it.

command	tests	test time
`cargo test -p hydrofrac --features cross-check`	182 passed, 0 failed (plus 2 ignored doc tests)	111 s
`cargo test -p hydro_std --lib hydrofrac`	12 passed, 0 failed, 3 ignored	66 s
`cargo test -p hydro_test --lib cluster::ec_`	63 passed, 0 failed, 10 ignored	564 s
the 10 teaching examples (§5.2), `--test-threads=1`	10 passed	62.5 s, mostly building and loading the compiled simulations (the first test waited 34 s for a trybuild compile); each `run` in them took under 25 ms

For comparison, implies §7 measured 103 s, 70 s (overlapping another suite) and 525 s for the same three suites. The slow tests in `cluster::ec_` (over 60 s each in this run) are G-Set gossip (`ec_demo_g_set_gossip_all_labels`, `ec_demo_g_set_gossip_certifies`), multi-Paxos (`ec_demo_multi_paxos_all_labels`), the uniform-broadcast demo, CTP-3 with a recurring timer, and three brute-force cross-checks in `ec_bruteforce` (`brute_force_agrees_on_a_plain_broadcast_with_three_payloads`, `brute_force_agrees_on_two_pc_cooperative_termination`, `brute_force_finds_the_two_pc_cooperative_termination_violation`). The cross-checks are slow because the oracles enumerate every schedule with no budget. The demos are slow largely because of the schedule budget (§7.5): each fault set of a program with many schedules enumerates up to 40,000 sent messages before it falls back to 8 samples (HANDOFF §5a measured this: gossip went from 43-64 s to 97-138 s, multi-Paxos from 38-39 s to 102 s, uniform broadcast from 5-9 s to 58 s, when the budget replaced the knob).

9.2 Races against the retired "lite" engine (not re-measured)

hydrofrac's earlier engine ("lite", retired because it was unsound: HANDOFF §2, S1 and S2) is no longer in the tree, so this comparison cannot be cheaply redone. It is quoted from `docs/design/races-vs-lite-comparison.md` (measured 2026-10-24 and 2026-10-25, at `664c60e2f1` and before), with a per-label print that is not committed. Since then the schedule budget replaced the sampling knob (`281245a80b`, which made some sampled tests exhaustive and changed their counts) and `implies` was added (whose own measurement found every pre-existing verdict and stats line unchanged, implies §7). So these numbers are indicative, not current.

Of 240 lite label rows, 232 were matched to the same label under races. Verdicts: 185 the same, 4 the same claim, 42 stronger (20 `Exhausted` verdicts became certificates; 22 counterexamples gained a certified tolerance), **0 weaker**, and 1 lite certificate became a counterexample (S1, lite's false certificate). Cost over the matched rows:

	lite	races
fault sets tried	1,340	2,856 (2.1×)
schedules run	85,051	59,647 (0.70×)

After item C (sequenced broadcasts and the crash-point tie-break), over 280 rows: fault sets 3,615 → 2,555, schedules 80,409 → 52,295. The rows still above 2× lite's cost, per HANDOFF §6 item 5: ABD phase 2 (8.4×), `reliable_broadcast_closed` `new_messages` (5-6×; the comparison file says item C resolved this group), G-Set gossip (about 4×), CTP-3 (3.2×), multi-Paxos `prepares` (8×). The usual cause is a label whose every derivation passes a fragile step: no clause protects any pair, and the solver proposes every fault set within the bounds.

10. Commands

All commands run from the repository root, on branch `hydrofrac-guide` (this guide on top of `b169662e51`, which is `hydrofrac-implies` and `origin/hydrofrac-races`). They are safe to paste into zsh: no inline comments, no unquoted globs. Never set `BOLERO_RANDOM_TEST_TIME_MS`; the check refuses to run with it. Diagrams need Graphviz's `neato` on the `PATH` (on macOS: `brew install graphviz`).

Build everything the suites need:

```
cargo check -p hydrofrac -p hydro_test -p hydro_std --all-targets --features hydrofrac/cross-check
```

The five label examples, one at a time, with reports printed:

```
cargo test -p hydro_test --lib -- cluster::ec_teaching:: --test-threads=1 --nocapture
```

The five `implies` examples:

```
cargo test -p hydro_test --lib -- cluster::ec_implies_teaching:: --test-threads=1 --nocapture
```

The examples together with their brute-force regression tests (`ec_teaching_checks.rs`, `ec_implies_checks.rs`):

```
cargo test -p hydro_test --lib -- cluster::ec_teaching cluster::ec_implies --test-threads=1 --nocapture
```

One example by name:

```
cargo test -p hydro_test --lib -- cluster::ec_teaching::one_lost_message_leaves_a_receiver_without_the_item --
nocapture
```

The three suites (§9 has their times):

```
cargo test -p hydrofrac --features cross-check
cargo test -p hydro_std --lib hydrofrac
cargo test -p hydro_test --lib cluster::ec_
```

The round-3 holes (each test passes while its hole is open):

```
git checkout hydrofrac-review3-a
cargo test -p hydrofrac --features cross-check review3a
git checkout hydrofrac-review3-b
cargo test -p hydrofrac --features cross-check review3b
```

If a build fails with "colliding StableCrateId", remove the staged copy and rebuild (HANDOFF §7):

```
rm -rf target/hydro_trybuild/hydrofrac
```

Diagrams. List every focused diagram the last runs wrote:

```
find target/hydrofrac -name counterexample.svg
```

Open one on macOS (Safari renders the SVG; Quick Look, `qlmanage -p`, also works):

```
open -a Safari target/hydrofrac/a_sender_crash_mid_broadcast_leaves_a_receiver_without_the_item/
received_ec_teaching.rs/counterexample.svg
```

Open every focused diagram of one test:

```
find target/hydrofrac/two_redundant_senders_survive_one_crash_but_not_two -name counterexample.svg -print0 | xargs -0
open -a Safari
```

Render a `.dot` by hand (when `neato` was missing during the run), as SVG or PNG:

```
neato -n2 -Tsvg counterexample.dot -o counterexample.svg
neato -n2 -Tpng counterexample.dot -o counterexample.png
```

With `CARGO_TARGET_DIR` set, replace `target` above by its value.

This guide's PDF is rendered with:

```
cd docs/design
pandoc hydrofrac-guide.md -o hydrofrac-guide.pdf --pdf-engine=typst --include-before-body=hydrofrac-guide-style.typ
```